Procedural walk animation 2D Documentation v1.1

Table of contents:

Important note on package dependencies 2
Demo Scenes 2
Procedural Animation setup 3
Procedural animation script settings 5
Legs Settings tab 5
Body setting Tab 9
Step effects tab 11
Technical documentation 12

Important note on package dependencies

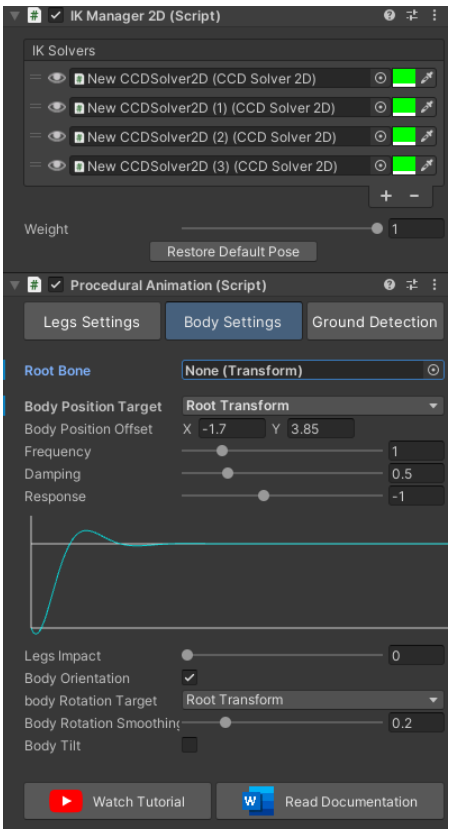
To run demo scene, you need to install **2D animation, 2D PSD importer** and **Cinemachine.**

Demo Scenes

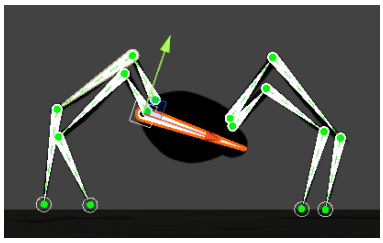
in Assets/ Procedural Walk Animation 2D/Scenes/ you will find a few demo scenes. Here you can see procedural animations with lots of different settings and presets. All of these were created by me, so you can freely use them in your own projects!

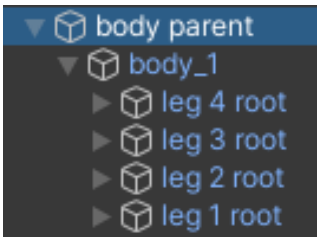
Procedural Animation setup

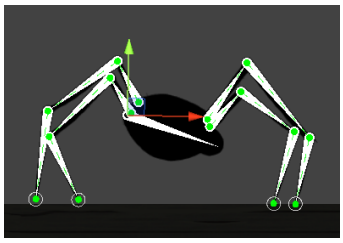
- Import your rigged sprite in unity (here is a great YouTube tutorial that I always use - https://www.youtube.com/watch?v=5hF4DBXpHac&t=304s).
- After you successfully set up the IK, add the Procedural Animation component to the same object as the IKManager2D (both should be attached to the root transform of your character)



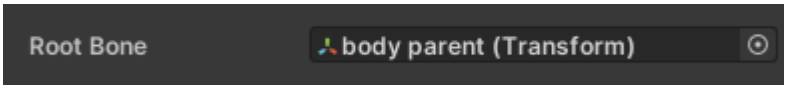
- Select the root bone of your character (body bone) and check its orientation. If the Y-axis (green arrow) is not pointing up and your body is slightly rotated (see Screenshot 1), you should select it, right-click, and choose "Create Empty Parent." You can rename it to "bodyParent" (see Screenshot 2). **The body parent should now have a green arrow pointing straight up** (see Screenshot 3).



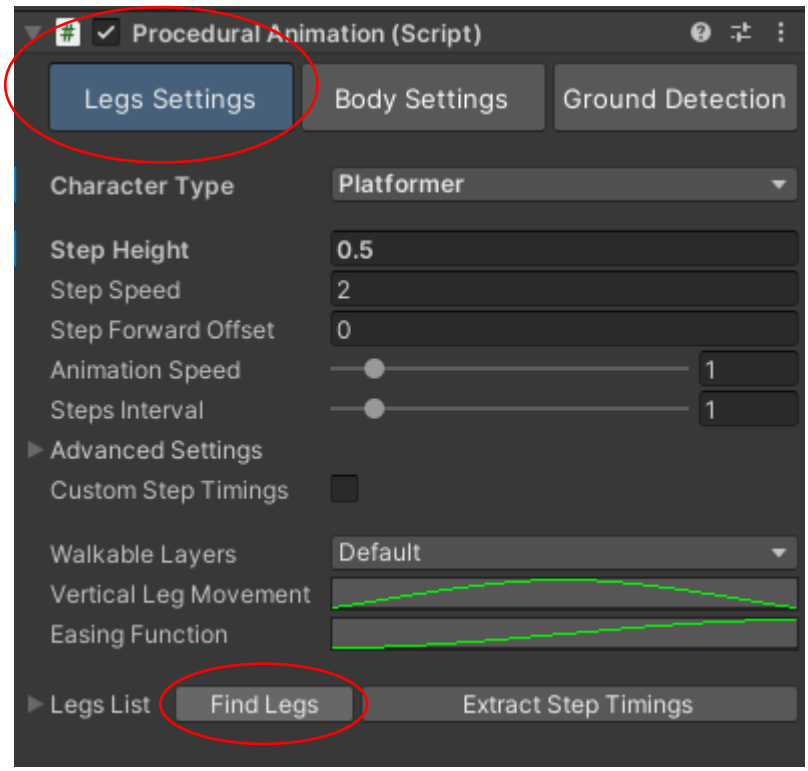




- Select the new body parent and assign it to the "Root Bone" field. When you do this, you should see that the buttons below, which lead to this documentation and the YouTube tutorial, have disappeared.



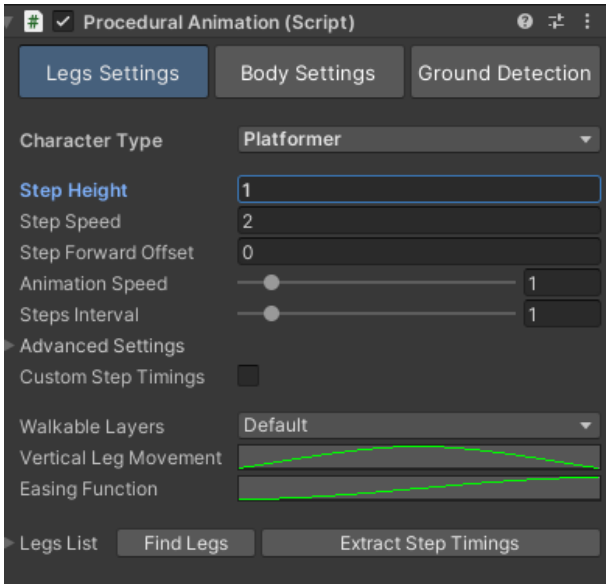
- Last step is to select the Legs Setting tab and click “find legs” button. **You should do this after you set up the IK and with IkManager2D component attaches to the same object and procedural animation one!**



Congratulations! You’ve successfully completed the setup! If you now enter play mode, you should see the animation in action when you move your character around. If not, **make sure that the surface your character walks on has colliders**. If body of your model is at the wrong place change body position offset on the body setting tab.

Procedural animation script settings

Legs Settings tab



- **Character Type** The platformer character type uses raycasts to determine the foot positions underneath it, while the top-down character type moves IK targets to default positions and places them on nearby walls when available. Use the platformer type if your game view is from the side, and select the top-down type if your game features a top-down view.

- **Step Height** defines how high in units the leg will move during the step

- **Step Speed** defines speed of the leg movement during the step

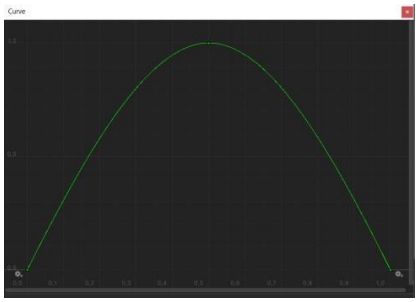
- **Step Forward Offset** defines how far forward will the leg move during the step

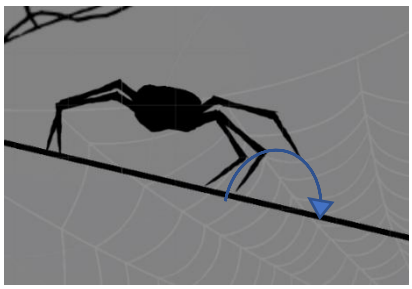
- **Animation Speed and Steps Interval**. In the update method there is cycle that calls make step function every second. The duration of the cycle is controlled by **Steps Interval** (if you set it to two your character will make step every two seconds) **animation speed** controls the overall speed of the animation.

- **Custom Step Timings**. If you want specific timings for your animation, use this option. Each slider represents an offset in seconds for taking a step. For example, setting the value to 0.5 means that the leg will take a step only half a second after the start of the animation.

- Set every walkable surface in **layer mask**.

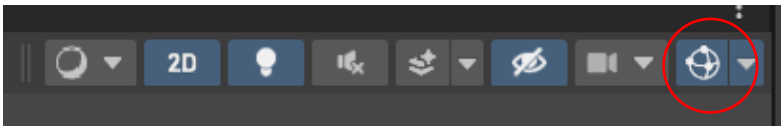
- **Vertical leg movement** is path of the leg on Y axis during the step (should start at 0 with value of 0 and end at one with value of 0).



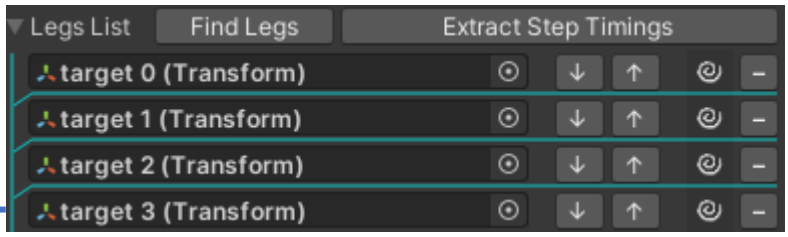


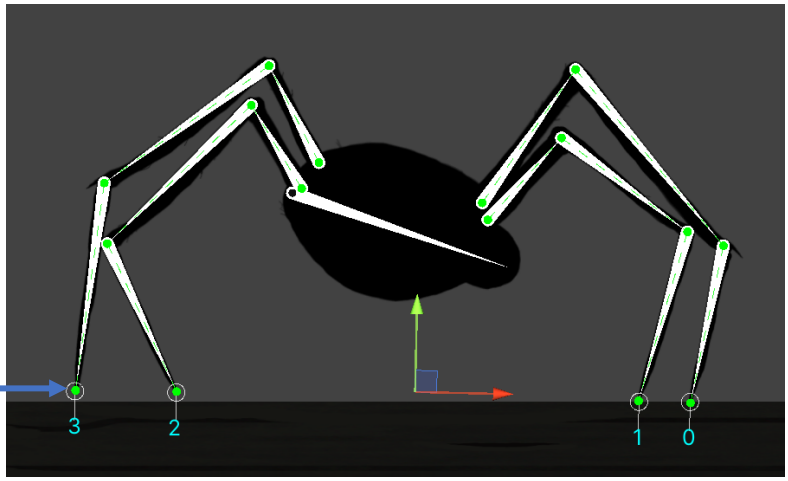
- **Easing Function** represents the speed of each leg animation. By default legs move slowly at first than fast and then slowly again towards the end of the animation. (it should start at 0 with value of 0 and end at 1 with value of 1).

- **Legs List** is the list of all IK targets of our character. During the setup when we clicked Find Legs button it automatically found all the IK targets from the IkManager2D and renamed them to leg1, leg2, etc. When we click on Leg list property, we can see all the leg IK targets and some UI icons next to it. The order of this list represents the order in which each leg will take a step. By pressing up and down arrow buttons on the right you can change the legs moving order. All the targets names have a number for example target 0. To easily understand which target belongs to which leg turn on the gizmos at the top right corner of the scene view. After that you should be able to see the numbers under your character feet that represent the target from the Legs list.



Toggle gizmos to see the leg targets indexes

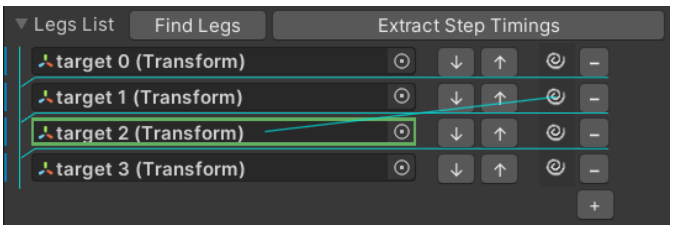




Each leg in the Legs List has a number in its name to easily identify that leg on your character you can toggle Gizmos and see the same number under the corresponding foot.

By default each leg will take a step after another. The blue lines between targets in the Legs List represent just that.

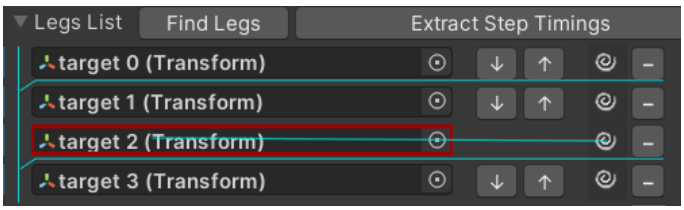
By pressing on the lasso icon and holding our mouse we can see that the blue line began following our cursor. That means that we now can link two legs together and they will move at the same time. By moving our cursor over other target in the legs list while still holding the left mouse button we can see a green outline appear on top of the leg that we want to link.



If we let go on the mouse now, we will link two legs together and they will now move at the same time. We can clearly see that by looking at our leg list and noticing that the line between two legs has disappeared.

Leg 1 and Leg 2 will now move together

If you accidentally linked a wrong leg, you can make it move separately again by linking it to itself. When you move the cursor over the same leg it will draw a red outline which indicates that this leg will no longer be linked to others



Leg 2 will now move separately again. We can see that by the line that had appeared between target 1 and target 2.

This feature is extremely useful to quickly create a new walking pattern. For example, by linking target 0 to target 1 and target 3 to the target 2 we can create a gallop movement. Note that for this character targets 0 and 1 are front legs and 2,3 are back legs for you it might be something else.

We can also change this groups moving order by pressing arrows just like it was shown before with individual legs.

Lastly the “-” and “+” buttons on the right are used to add or remove legs from the legs list. **Note that if you add a new leg via “+” button you have to assign it yourself and the number of the target won’t be shown in the scene view like with other legs. It only works when you click “find legs” button like we did before.**

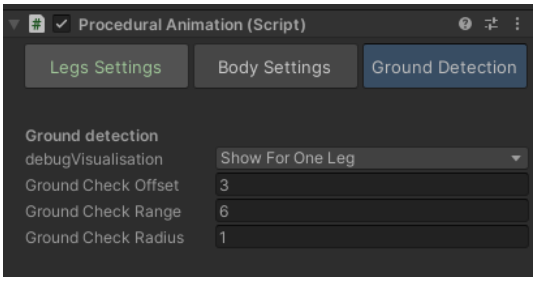
Advanced settings

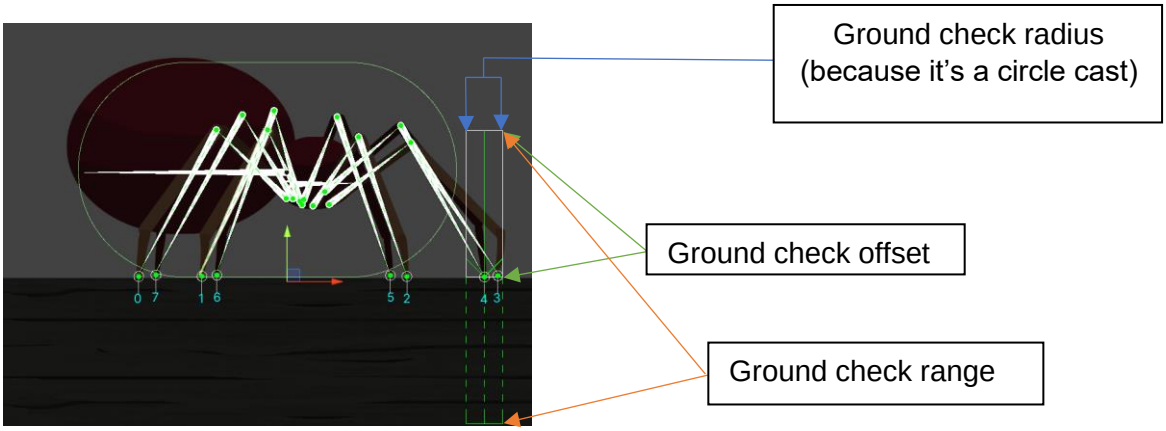
Click on the advanced settings foldout to reveal a few more parameters (movement thresholds and ground detection)

- **Minimum distance to make step** this parameter is pretty self-explanatory it's the minimum distance that the root transform needs to move before the leg can make a step. I don't recommend setting this option lower than default value unless you know what you're doing because the character will take a lot of small steps on sudden movements which can be bad if you want to add audio or visual effects to the animation. (sense every time character makes a step the **OnStepFinished** event is triggered and it can get messy)

- **Maximum distance to make step** on the other hand if our character moved to fast or too far we don't want to wait till it moves each leg one by one to the new position. This parameter controls what distance is required for legs to ignore its timings and move to a new position instantly. However if you set it too low it can ruin the animation looks because leg will ignore the step timings which can lead to a bad looking animation. Generally this option is great if your character moves really fast and the animation does not keep up or if your character is teleported somewhere and you don't want to wait for its legs to move to the new position one by one.

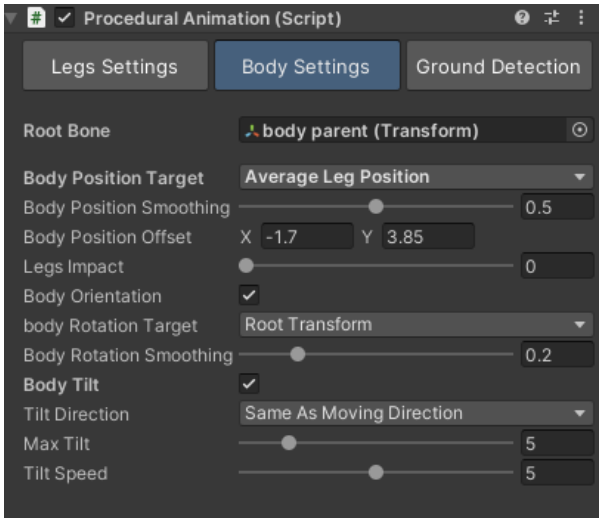
- **Ground detection** Ground detection settings are pretty strait forward if you enable debug visualization you can clearly see what each option is doing. A procedural animation script shoots a cirlecast from the default leg position to the ground to determine where to place its foot. And all of this options are controlling the range of this circlecast.



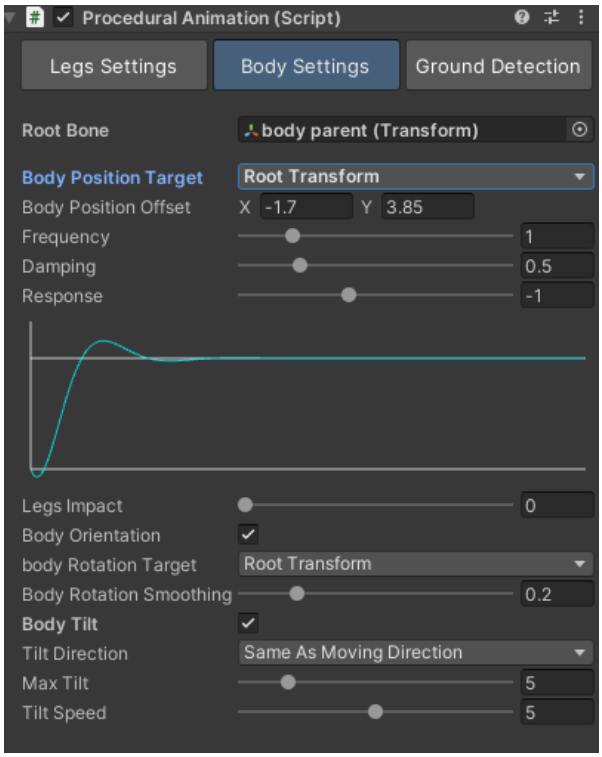


Body setting Tab

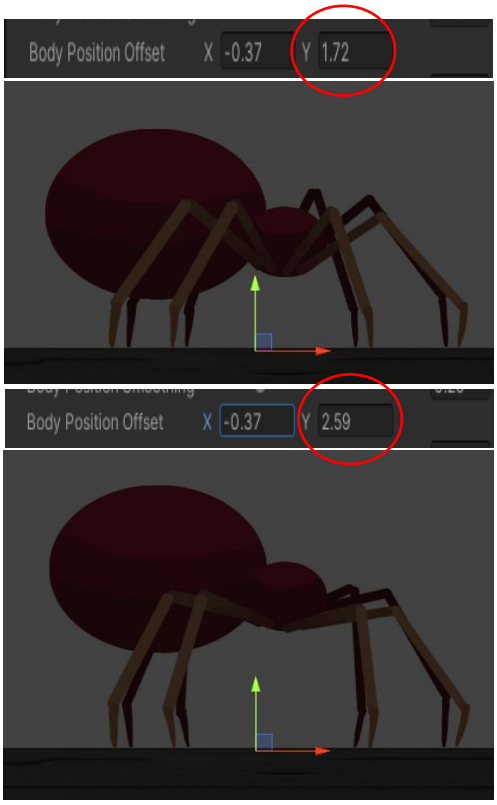
- **Body position target**
  Average leg position option calculates the body position based on the average leg positions (center of mass), which is great for more dynamic animations. For example, insects or robots are a great fit for this option. The second option will smoothly interpolate towards the root transform of the model, as represented by the graph below that appears when you chose it. The blue line on the graph represents the movement of the body when you move the character.



- **Body position smoothing**
  When choosing the average leg position as a body position target this option will appear. This slider represent how smoothly body will interpolate towards the center of mass. 0 meaning it will instantly snap to the new position and 1 meaning it will interpolate really slow.



- **Body position offset**
  Use it to change default body position. Example:



- If **body orientation** is enabled body will rotate based on an average legs position.

- **Body rotation target**
  Root transform is a great option for more uneven terrains because it rotates the whole model based on the leg positions. This also allows the model to walk upside down since the raycast direction for checking the ground will also change. The Root bone option rotates only the body bone, which is ideal if you want your character to move more evenly, as this won't change the raycast direction for ground detection. The feet will always choose target step points right under them.

- **Body rotation smoothing** works just like body position smoothing 0 means body will rotate instantly and 1 means it will rotate really smoothly.

- **Body tilt** when this option is enabled the script will slightly rotate the body towards or away from moving direction.

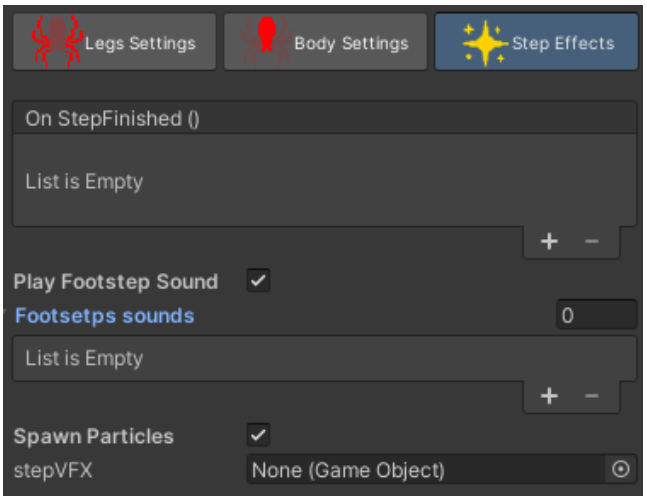
- **Tilt direction** Regular option means that the body will rotate towards the direction of movement, while Inverse means it will rotate in the opposite direction.

- **Max tilt** is how much in degrees the body will rotate while moving.

- **Tilt speed** is how fast the body will rotate.

Step effects tab

- **On StepFinished ()** is a simple unity event that Invokes every time the leg makes a step.
- **Play Footstep sound** plays a random sound from the sound clips that you add every time the leg makes a step.
- **Spawn particles** spawns a particles like dust every time the leg finishes a step on that position.



OnStepFinishedEvent is another event in the procedural animation script this event is not displayed in the inspector as the first one and is used for a more complex logic. For example: your character is really big and to feel it you want to add a camera shake every time it takes a step. A good way to do this is implement this logic in some other script. First you need a reference to the procedural animation of your character than you need to subscribe to the event like shown below.

```
using UnityEngine;

namespace LolopupkaAnimations2D
{
0 references
public class StepEffects : MonoBehaviour
{
    2 references
    private proceduralAnimation proceduralAnimation;
    0 references
    void Start()
    {

        if (TryGetComponent<proceduralAnimation>(out proceduralAnimation))
        {
            proceduralAnimation.OnStepFinished += ProceduralAnimation_OnStepFinished;
        }
        else
        {
            Debug.LogError("procedural animation script required on " + gameObject);
        }
    }

    1 reference
    private void ProceduralAnimation_OnStepFinished(object sender, Vector3 LegPosition)
    {
        // your custom logic
        // for example you can add a camera shake everytime character takes a step
    }
}

}
```

Technical documentation

For more details about how the different Scripts work, please refer to the comments within the Scripts themselves. If you have any specific question, you can contact me any time at karen.magik.user@gmail.com or at my discord server https://discord.gg/7GXeUtvGgY

Good luck with your projects! :)